



UNIVERSITÉ DE KINSHASA
FACULTÉ DES SCIENCES ET TECHNOLOGIES
DÉPARTEMENT DE MATHÉMATIQUES, STATISTIQUE ET
INFORMATIQUE
MENTION INFORMATIQUE, M1 LMD
FILIÈRE DATA SCIENCE ET INTELLIGENCE
ARTIFICIELLE

TRAVAIL DU COURS DE DATA SCIENCE ET BIG DATA

SUJET
« ALGORITHME DE NUÉES
DYNAMIQUES »

Travail fait par l'étudiant :

BUETUENA MALOZI Jordi

Travail dirigé par :

Le Professeur **KAFUNDA Pierre**
et le Doctorant **KAMINGU Gradi**

ANNÉE ACADÉMIQUE : 2025–2026

Table des matières

1. Introduction sur le Machine Learning	1
1.1. Contexte historique	1
1.2. Définitions et paradigmes	1
1.3. Le clustering et sa place	1
2. Algorithme de Nuées Dynamiques	3
2.0. Présentation générale	3
2.1. Fondement mathématique	3
2.2. Implémentation en Python	4
3. Applications de l'algorithme de Nuées Dynamiques	13
3.1. Application en neurologie (Diday, 1971)	13
3.2. Domaines d'application modernes	13
3.3. Extensions et algorithmes dérivés	14
4. Forces et faiblesses	15
4.1. Forces	15
4.2. Faiblesses	15
5. Conclusion	17
6. Références bibliographiques	18
7. Annexe : illustration d'exécution	19

1. Introduction sur le Machine Learning

1.1. Contexte historique

Le **Machine Learning** (apprentissage automatique) désigne l'ensemble des méthodes algorithmiques permettant à une machine d'*apprendre à partir de données* sans être explicitement programmée pour chaque tâche. Historiquement, cette discipline émerge dans les années 1950–1960 avec les travaux d'Arthur Samuel sur le jeu de dames (1959), le perceptron de Rosenblatt (1958) et les premières réflexions sur la reconnaissance des formes. Le terme lui-même est popularisé par Arthur Samuel, tandis que la formalisation théorique viendra plus tard avec Valiant (PAC learning, 1984) et Vapnik (SVM, 1995).

Le Machine Learning connaît une explosion spectaculaire depuis les années 2010, portée par trois facteurs : (i) la disponibilité de *big data*, (ii) la puissance de calcul des GPU, et (iii) les progrès des réseaux de neurones profonds (*deep learning*).

1.2. Définitions et paradigmes

Selon Tom Mitchell (1997), un programme *apprend* à partir d'une expérience E par rapport à une tâche T et une mesure de performance P , si sa performance sur T , mesurée par P , s'améliore avec E .

On distingue trois grands paradigmes :

- **Apprentissage supervisé** : les données sont étiquetées (x_i, y_i) ; on cherche une fonction $f : X \rightarrow Y$ qui généralise (régression, classification).
- **Apprentissage non supervisé** : aucune étiquette ; on cherche à *découvrir une structure cachée* dans les données (clustering, réduction de dimension, détection d'anomalies).
- **Apprentissage par renforcement** : un agent apprend par interaction avec un environnement, guidé par une fonction de récompense.

1.3. Le clustering et sa place

Le **clustering** (classification automatique ou non supervisée) consiste à regrouper des individus en classes homogènes sans connaissance préalable des étiquettes. C'est un problème fondamental dont l'origine remonte aux travaux de Sneath, Sokal, et à l'école française d'analyse des données (Benzécri, Diday).

L'algorithme des **k-means** (MacQueen, 1967) est la méthode la plus connue, mais il présente une limite majeure : chaque classe est représentée par un *unique point* (son

centroïde). Cette représentation appauvrit la structure d'une classe qui peut être allongée, courbe ou multimodale.

C'est précisément pour dépasser cette limite qu'Edwin Diday propose en 1971 la **méthode des Nuées Dynamiques**, objet central de ce travail.

2. Algorithme de Nuées Dynamiques

2.0. Présentation générale

La méthode des **Nuées Dynamiques**, introduite par Edwin Diday dans la *Revue de Statistique Appliquée* (vol. XIX, n°2, 1971, pp. 19–33), est une généralisation de la méthode des k-means. Au lieu de représenter chaque classe par un simple centroïde, on la représente par une *nuée*, c'est-à-dire une structure représentative plus riche : un ensemble d'étalons, un axe factoriel, une distribution de probabilité, ou tout autre objet adapté au problème.

Formellement, on se donne :

- $E = \{x_1, \dots, x_N\}$: l'ensemble des individus à classer ;
- K : le nombre de classes souhaitées ;
- P_1 : l'ensemble des parties de E ayant n_1 objets ;
- $L^{(n)} = (E_1^{(n)}, \dots, E_K^{(n)})$ avec $E_k^{(n)} \in P_1$: la nuée de la k -ième classe à l'itération n .

L'algorithme alterne deux étapes :

1. **Affectation** : chaque individu $x \in E$ est affecté à la classe C_i dont la nuée E_i minimise la distance $D(x, E_i)$:

$$C_i^{(n)} = \left\{ x \in E \mid D(x, E_i^{(n)}) < D(x, E_j^{(n)}), \forall j \neq i \right\}.$$

2. **Mise à jour** : chaque nuée est recalculée à partir de ses membres via une fonction d'agrégation-écartement $R(x, i, L)$:

$$E_i^{(n+1)} = \text{les } n_1 \text{ objets de } E \text{ qui minimisent } R(x, i, L^{(n)}).$$

On itère jusqu'à ce que la partition et les nuées se stabilisent (convergence).

2.1. Fondement mathématique

2.1.1. Notations

On note $\mathcal{P}(E)$ l'ensemble des parties de E , et $\mathbb{R}_+$ l'ensemble des réels positifs. La fonction

$$D : E \times \mathcal{P}(E) \rightarrow \mathbb{R}_+$$

mesure la « distance » d'un individu à une nuée. La fonction

$$R : E \times \{1, \dots, K\} \times L_K \rightarrow \mathbb{R}_+$$

est appelée *fonction d'agrégation-écartement*.

2.1.2. Choix de la fonction D

Selon le type de nuée, on utilise :

— **Nuée ponctuelle** (cas k-means) :

$$D(x, E_i) = \|x - \mu_i\|_2, \quad \mu_i = \frac{1}{|C_i|} \sum_{y \in C_i} y.$$

— **Ensemble d'étalons** $E_i = \{e_1, \dots, e_{n_1}\}$:

$$D(x, E_i) = \frac{1}{n_1} \sum_{j=1}^{n_1} \|x - e_j\|_2.$$

— **Axe factoriel / sous-espace affine** $E_i = (\mu_i, V_i)$ avec V_i base orthonormée de dimension p :

$$D(x, E_i) = \left\| (x - \mu_i) - \sum_{v \in V_i} \langle x - \mu_i, v \rangle v \right\|_2.$$

— **Distribution gaussienne** $E_i = (\mu_i, \Sigma_i)$:

$$D(x, E_i) = \sqrt{(x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i)} \quad (\text{distance de Mahalanobis}).$$

2.1.3. Convergence

Diday définit la suite

$$S(L, L^{(0)}) = \sum_{k=1}^K \sum_{x \in E_k} R(x, k, L^{(0)})$$

et la suite auxiliaire U_p :

$$U_0 = S(L^{(0)}, L^{(0)}), \quad U_{2n} = S(L^{(n)}, L^{(n)}), \quad U_{2n-1} = S(L^{(n)}, L^{(n-1)}).$$

Définition (fonction carrée). R est dite carrée sur E si

$$S(L, M) \leq S(M, M) \implies S(L, L) \leq S(L, M).$$

Théorème (Diday, 1971). *Si R est carrée, la suite $L^{(n)}$ converge et sa limite est un optimum local, c'est-à-dire une configuration où aucune nuée ne peut être améliorée par un autre choix d'étalons.*

La démonstration repose sur trois propositions intermédiaires : (i) la suite U_n est décroissante, (ii) si $L^{(n)}$ converge, sa limite est un optimum local, (iii) la convergence de U_n implique celle de $L^{(n)}$.

En pratique, on observe que la méthode converge même lorsque R n'est pas carrée, ce qui explique son succès applicatif.

2.2. Implémentation en Python (from scratch)

L'implémentation suivante utilise exclusivement le module `math` et `random` de la bibliothèque standard Python : aucune bibliothèque externe (numpy, scipy, sklearn) n'est requise. Elle gère les cinq types de nuées prévus.

2.2.1. Code source complet

```

1  # -*- coding: utf-8 -*-
2  import math
3  import random
4
5  # 1. Opérations vectorielles / matricielles de base
6  def v_add(u, v): return [a + b for a, b in zip(u, v)]
7  def v_sub(u, v): return [a - b for a, b in zip(u, v)]
8  def v_scale(c, v): return [c * x for x in v]
9  def v_dot(u, v): return sum(a * b for a, b in zip(u, v))
10 def v_norm(v): return math.sqrt(sum(x * x for x in v))
11 def v_dist(a, b): return v_norm(v_sub(a, b))
12
13
14 def v_mean(vectors):
15     n, d = len(vectors), len(vectors[0])
16     return [sum(v[j] for v in vectors) / n for j in range(d)]
17
18
19 def v_cov(vectors, mu=None):
20     """Matrice de covariance (estimateur 1/(n-1))."""
21     n, d = len(vectors), len(vectors[0])
22     if mu is None:
23         mu = v_mean(vectors)
24     C = [[0.0] * d for _ in range(d)]
25     for v in vectors:
26         diff = v_sub(v, mu)
27         for i in range(d):
28             for j in range(d):
29                 C[i][j] += diff[i] * diff[j]
30     denom = max(1, n - 1)
31     for i in range(d):
32         for j in range(d):
33             C[i][j] /= denom
34     return C
35
36
37 def mat_inv(A):
38     """Inverse par Gauss-Jordan (avec regularisation si singulier)."""
39     n = len(A)
40     M = [row[:] + [1.0 if i == j else 0.0 for j in range(n)]
41          for i, row in enumerate(A)]
42     for i in range(n):
43         piv = max(range(i, n), key=lambda k: abs(M[k][i]))
44         if abs(M[piv][i]) < 1e-12:
45             M[i][i] += 1e-6
46         continue
47     M[i], M[piv] = M[piv], M[i]
48     p = M[i][i]
49     for j in range(2 * n):
50         M[i][j] /= p
51     for k in range(n):
52         if k != i:
53             f = M[k][i]
54             for j in range(2 * n):
55                 M[k][j] -= f * M[i][j]
56     return [row[n:] for row in M]

```

```

57
58
59 def jacobi_eig(A, max_iter=200, tol=1e-12):
60     """Diagonalisation de Jacobi d'une matrice symétrique."""
61     n = len(A)
62     A = [row[:] for row in A]
63     V = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
64     for _ in range(max_iter):
65         off, p, q = 0.0, 0, 1
66         for i in range(n):
67             for j in range(i + 1, n):
68                 if abs(A[i][j]) > off:
69                     off = abs(A[i][j]); p, q = i, j
70             if off < tol:
71                 break
72         theta = (A[q][q] - A[p][p]) / (2 * A[p][q])
73         t = (1 if theta >= 0 else -1) / (abs(theta) + math.sqrt(theta * theta +
74                                     1))
75         c = 1 / math.sqrt(t * t + 1)
76         s = t * c
77         for i in range(n):
78             if i != p and i != q:
79                 aip, aiq = A[i][p], A[i][q]
80                 A[i][p] = c * aip - s * aiq; A[p][i] = A[i][p]
81                 A[i][q] = s * aip + c * aiq; A[q][i] = A[i][q]
82                 app, aqq, apq = A[p][p], A[q][q], A[p][q]
83                 A[p][p] = c * c * app - 2 * s * c * apq + s * s * aqq
84                 A[q][q] = s * s * app + 2 * s * c * apq + c * c * aqq
85                 A[p][q] = A[q][p] = 0.0
86             for i in range(n):
87                 vip, viq = V[i][p], V[i][q]
88                 V[i][p] = c * vip - s * viq
89                 V[i][q] = s * vip + c * viq
90         vals = [A[i][i] for i in range(n)]
91         vecs = [[V[i][j] for i in range(n)] for j in range(n)] # vecs[j] = j-
92                             me colonne
93
94
95 # 2. Classe principale : Nuees Dynamiques
96 class NueeDynamique:
97     """
98     data          : liste de vecteurs (individus)
99     K              : nombre de classes
100     nuee_type     : 'point' | 'set' | 'axes' | 'distribution' | 'structure'
101     n1            : nombre d' talons (uniquement pour 'set')
102     p             : dimension du sous-espace (pour 'axes'/'structure')
103     """
104
105     def __init__(self, data, K, nuee_type='point', n1=1, p=1):
106         self.data = data
107         self.N = len(data)
108         self.M = len(data[0])
109         self.K = K
110         self.nuee_type = nuee_type
111         self.n1 = min(n1, self.N)
112         self.p = min(p, self.M)

```

```

113 self.reps = [None] * K          # K n u e s (r e p r e s e n t a t i o n s)
114 self.classes = [[] for _ in range(K)]
115
116
117 # ----- D(x, E) : distance d'un point      une n u e -----
118 def dist_to_rep(self, x, rep):
119     if rep is None:
120         return float('inf')
121     t = self.nuee_type
122
123     if t == 'point':
124         return v_dist(x, rep)
125
126     if t == 'set':
127         if not rep:
128             return float('inf')
129         # distance moyenne aux talons (robuste)
130         return sum(v_dist(x, y) for y in rep) / len(rep)
131
132     if t in ('axes', 'structure'):
133         mu, basis = rep
134         diff = v_sub(x, mu)
135         for v in basis:
136             diff = v_sub(diff, v_scale(v_dot(diff, v), v))
137         return v_norm(diff)
138
139     if t == 'distribution':
140         mu, inv = rep
141         diff = v_sub(x, mu)
142         tmp = [sum(inv[i][j] * diff[j] for j in range(self.M))
143               for i in range(self.M)]
144         return math.sqrt(max(0.0, v_dot(diff, tmp)))
145
146     return float('inf')
147
148 # ----- Mise      jour d'une n u e      partir de ses membres
149 # -----
150 def compute_rep(self, class_idx):
151     if not class_idx:
152         return None
153     pts = [self.data[i] for i in class_idx]
154     t = self.nuee_type
155
156     # --- (1) Point unique : centro de ---
157     if t == 'point':
158         return v_mean(pts)
159
160     # --- (2) Ensemble d' talons      : n1 points de E minimisant R(x,i,L) ---
161     if t == 'set':
162         scores = []
163         for idx in range(self.N):
164             x = self.data[idx]
165             d = sum(v_dist(x, self.data[j]) for j in class_idx) / len(class_idx)
166             scores.append((d, idx))
167             scores.sort(key=lambda z: z[0])
168         return [self.data[idx] for _, idx in scores[:self.n1]]
169
170     # --- (3) / (5) Axes factoriels / sous-espace affine ---

```

```

170 if t in ('axes', 'structure'):
171     mu = v_mean(pts)
172     if len(pts) < 2:
173         return (mu, [])
174     cov = v_cov(pts, mu)
175     vals, vecs = jacobi_eig(cov)
176     order = sorted(range(len(vals)), key=lambda i: -vals[i])
177     p = min(self.p, self.M, len(order))
178     basis = [vecs[i] for i in order[:p]]
179     return (mu, basis)
180
181 # --- (4) Distribution de probabilit (gaussienne) ---
182 if t == 'distribution':
183     mu = v_mean(pts)
184     if len(pts) < 2:
185         cov = [[1.0 if i == j else 0.0 for j in range(self.M)]
186               for i in range(self.M)]
187     else:
188         cov = v_cov(pts, mu)
189         for i in range(self.M):
190             cov[i][i] += 1e-6
191         return (mu, mat_inv(cov))
192
193 return None
194
195 # ----- tape 2 de l'algorithme : partition -----
196 def assign(self):
197     cls = [[] for _ in range(self.K)]
198     for idx, x in enumerate(self.data):
199         best_k, best_d = 0, float('inf')
200         for k in range(self.K):
201             d = self.dist_to_rep(x, self.reps[k])
202             if d < best_d:
203                 best_d, best_k = d, k
204         cls[best_k].append(idx)
205     return cls
206
207 # ----- Crit re S(L, L) -----
208 def compute_S(self):
209     total = 0.0
210     for k in range(self.K):
211         c = self.classes[k]
212         if not c:
213             continue
214         for idx in c:
215             x = self.data[idx]
216             d = sum(v_dist(x, self.data[j]) for j in c) / len(c)
217             total += d
218     return total
219
220 # ----- Initialisation -----
221 def initialize_random(self):
222     idx = list(range(self.N))
223     random.shuffle(idx)
224     self.classes = [[] for _ in range(self.K)]
225     for i, j in enumerate(idx):
226         self.classes[i % self.K].append(j)
227     self.reps = [self.compute_rep(c) for c in self.classes]

```

```

228
229 def initialize_manual(self, initial_reps):
230     self.reps = initial_reps
231     self.classes = self.assign()
232
233     # ----- Boucle principale -----
234     def run(self, max_iter=100, verbose=True):
235         history = []
236         for it in range(max_iter):
237             old_classes = [c[:] for c in self.classes]
238             # tape 1 : affectation
239             self.classes = self.assign()
240             # tape 2 : mise jour des nu es
241             for k in range(self.K):
242                 if self.classes[k]:
243                     self.reps[k] = self.compute_rep(self.classes[k])
244             # Crit re
245             S = self.compute_S()
246             history.append(S)
247             if verbose:
248                 sizes = [len(c) for c in self.classes]
249                 print(f" It ration {it + 1:3d} : S = {S:.6f}    tailles = {sizes}")
250             # Test de convergence
251             if old_classes == self.classes:
252                 if verbose:
253                     print(f" >> Convergence atteinte    l'it ration {it + 1}")
254                 break
255             return history
256
257
258     # 3. Interface clavier
259     def input_int(prompt, min_val=None, max_val=None):
260         while True:
261             try:
262                 v = int(input(prompt))
263                 if min_val is not None and v < min_val:
264                     print(f" Valeur >= {min_val} attendue.")
265                     continue
266                 if max_val is not None and v > max_val:
267                     print(f" Valeur <= {max_val} attendue.")
268                     continue
269                 return v
270             except ValueError:
271                 print(" Entr e invalide. Entrez un entier.")
272
273
274     def input_vector(prompt, d):
275         while True:
276             s = input(prompt)
277             try:
278                 vals = [float(x) for x in s.replace(',', ' ').split()]
279                 if len(vals) != d:
280                     print(f" Il faut exactement {d} valeurs.")
281                     continue
282                 return vals
283             except ValueError:
284                 print(" Entr e invalide. Entrez des nombres s par s par des espaces.
285                        ")

```

```

285
286 # 4. Programme principal
287
288
289 def main():
290     print()
291     print(" ")
292     print(" M THODE DES NU ES DYNAMIQUES (E. Diday, 1971)")
293     print(" ")
294
295     # ----- Donn es -----
296     print("\n--- Saisie des donn es ---")
297     N = input_int("Nombre d'individus N : ", 2)
298     M = input_int("Nombre de param tres (dimension) M : ", 1)
299     print(f"Entrez les {N} individus ({M} valeur(s) par individu) :")
300     data = [input_vector(f" x{i + 1} : ", M) for i in range(N)]
301
302     # ----- Param tres -----
303     print("\n--- Param tres de l'algorithme ---")
304     K = input_int(f"Nombre de classes K (2 <= K <= {N}) : ", 2, N)
305
306     print("\nType de nu e (repr sentation des classes) :")
307     print(" 1. Point unique ( equivalent k-means)")
308     print(" 2. Ensemble de points (nu e d' talons)")
309     print(" 3. Axes factoriels (1 axe principal)")
310     print(" 4. Distribution de probabilit (gaussienne)")
311     print(" 5. Structure repr sentative (sous-espace affine)")
312     t = input_int("Votre choix : ", 1, 5)
313
314     if t == 1:
315         nuee_type, n1, p = 'point', 1, 1
316     elif t == 2:
317         nuee_type = 'set'
318         n1 = input_int(f"Nombre d' talons n1 par classe (1 <= n1 <= {N}) : ",
319             1, N)
320         p = 1
321     elif t == 3:
322         nuee_type, n1, p = 'axes', 1, 1
323     elif t == 4:
324         nuee_type, n1, p = 'distribution', 1, 1
325     else:
326         nuee_type = 'structure'
327         p = input_int(f"Dimension du sous-espace (1 <= p <= {M}) : ", 1, M)
328         n1 = 1
329
330     print("\nMode d'initialisation :")
331     print(" 1. Al atoire (recommand )")
332     print(" 2. Avec graine fixe (reproductible)")
333     seed = input_int("Votre choix : ", 1, 2)
334     if seed == 2:
335         random.seed(12345)
336
337     # ----- Ex cution -----
338     print("\n--- Ex cution de l'algorithme ---")
339     algo = NueeDynamique(data, K, nuee_type, n1=n1, p=p)
340     algo.initialize_random()
341     history = algo.run(max_iter=100, verbose=True)

```

```

342 # ----- R sultats -----
343 print("\n" + " ")
344 print("  R SULTATS")
345 print("")
346 for k in range(K):
347     print(f"\n--- Classe {k + 1} : {len(algo.classes[k])} individu(s) ---")
348     print(f"  Indices : {[i + 1 for i in algo.classes[k]]}")
349     rep = algo.reps[k]
350     if rep is None:
351         print("  (classe vide)")
352         continue
353     if nuee_type == 'point':
354         print(f"  Centre : {[round(v, 4) for v in rep]}")
355     elif nuee_type == 'set':
356         print(f"  {len(rep)}  talon (s) :")
357         for e in rep:
358             print(f"    {[round(v, 4) for v in e]}")
359     elif nuee_type in ('axes', 'structure'):
360         mu, basis = rep
361         print(f"  Centre : {[round(v, 4) for v in mu]}")
362         print(f"  Base du sous-espace ({len(basis)} vecteur(s)) :")
363         for v in basis:
364             print(f"    {[round(x, 4) for x in v]}")
365     elif nuee_type == 'distribution':
366         mu, inv = rep
367         print(f"  Moyenne : {[round(v, 4) for v in mu]}")
368         print(f"  Matrice de covariance inverse :")
369         for row in inv:
370             print(f"    {[round(x, 4) for x in row]}")
371
372     print(f"\nValeur finale S(L,L) = {history[-1]:.6f}")
373     print(f"Nombre d'it rations : {len(history)}")
374
375
376 if __name__ == "__main__":
377     try:
378         main()
379     except (KeyboardInterrupt, EOFError):
380         print("\n\nInterrompu par l'utilisateur.")

```

Listing 1 – Implémentation from scratch de l'algorithme des Nuées Dynamiques (Diday, 1971).

2.2.2. Organisation du code

Le code se structure en quatre blocs :

1. **Opérations vectorielles et matricielles** : additions, produits scalaires, normes, moyenne, covariance, inversion de matrice (Gauss-Jordan), diagonalisation de Jacobi pour l'ACP.
2. **Classe NueeDynamique** : implémente l'algorithme sous forme orientée-objet. Méthodes clés : `dist_to_rep` (calcule $D(x, E_i)$ selon le type de nuée), `compute_rep` (met à jour la nuée), `assign` (étape d'affectation), `compute_S` (critère global), `run` (boucle principale).
3. **Interface clavier** : saisie contrôlée des individus et des paramètres (`input_int`, `input_vector`).

4. **Programme principal** : dialogue avec l'utilisateur et affichage des résultats.

2.2.3. Choix d'implémentation notables

- La distance à une nuée d'étalons utilise la *moyenne* des distances aux étalons, version robuste du critère $\sum_{y \in E_i} d(x, y)$ du papier original.
- La sélection des n_1 étalons se fait sur **tout** E (pas seulement sur la classe), conformément à l'étape 3 de l'algorithme.
- La diagonalisation de Jacobi évite tout recours à numpy tout en restant numériquement stable.
- L'initialisation est aléatoire (uniforme sans remise), avec option de graine fixe pour la reproductibilité.

3. Applications de l'algorithme de Nuées Dynamiques

3.1. Application en neurologie (Diday, 1971)

Dans l'article fondateur, Diday applique la méthode à un problème médical concret fourni par le Docteur Marc-Vergnes. Il s'agissait de déterminer si 25 paramètres caractéristiques du *métabolisme énergétique du cerveau* permettaient de prévoir l'évolution de malades vasculaires-cérébraux.

Sur 70 individus, la méthode a mis en évidence **trois groupes** :

- Groupe I : forme « forte » (stable au ré-échantillonnage) ;
- Groupes II et III : formes « faibles », qui ont tendance à se mélanger lorsque l'on change les étalons initiaux.

Les résultats se recoupent exactement avec ceux d'une analyse hiérarchique, et permettent une prévision significative : dans le premier groupe, seuls 4 malades sont décédés, alors que dans les deux autres groupes on ne compte que 5 évolutions favorables.

3.2. Domaines d'application modernes

Depuis 1971, la méthode des Nuées Dynamiques a été appliquée avec succès dans de nombreux domaines :

- **Biologie et médecine** : classification de tumeurs à partir de données d'expression génique (microarrays), segmentation d'images médicales (IRM, scanners).
- **Marketing et CRM** : segmentation de clientèles, analyse RFM (Récence, Fréquence, Montant).
- **Documentation automatique** : classification de documents textuels (cf. travaux de Rocchio, Salton).
- **Reconnaissance des formes** : classification d'images satellites, reconnaissance de caractères manuscrits.
- **Sociologie et sciences humaines** : typologies d'enquêtes, analyse de données d'opinion.
- **Architecture et urbanisme** : classification de formes architecturales, analyse spatiale.
- **Bio-informatique** : regroupement de gènes co-exprimés, classification de protéines.

3.3. Extensions et algorithmes dérivés

La méthode des Nuées Dynamiques a inspiré plusieurs algorithmes célèbres :

- **k-means** : cas particulier où la nuée est un point unique.
- **ISODATA** (Ball & Hall, 1965) : version avec fusion et division dynamique de classes.
- **Fuzzy c-means** (Dunn, 1973 ; Bezdek, 1981) : affectation floue (degré d'appartenance).
- **Mixture of Gaussians / EM** : nuée = distribution gaussienne multivariée.
- **Learning Vector Quantization** (Kohonen) : variante supervisée avec prototypes multiples.
- **k-means++** (Arthur & Vassilvitskii, 2007) : amélioration de l'initialisation.

4. Forces et faiblesses

4.1. Forces

- **Généralité** : la méthode est un cadre unifié dont les k-means, les mélanges gaussiens et bien d'autres algorithmes sont des cas particuliers. L'utilisateur choisit la représentation de nuée adaptée à ses données.
- **Flexibilité de la distance** : on peut utiliser n'importe quelle distance D (euclidienne, χ^2 , Mahalanobis, similarités), ce qui permet de traiter des données hétérogènes (mixtes, textuelles, binaires).
- **Richesse de la représentation** : contrairement aux k-means, une classe peut être allongée, courbe ou multimodale — la nuée d'étalons joue le rôle de *squelette* ou d'*axe factoriel discret*.
- **Convergence prouvée** : sous l'hypothèse de fonction d'agrégation-écartement carrée, la convergence vers un optimum local est démontrée (théorème de Diday, 1971).
- **Efficacité computationnelle** : le calcul ne nécessite pas la matrice complète des distances deux à deux ($N(N-1)/2$), ce qui autorise le traitement de grands tableaux (jusqu'à plusieurs milliers d'individus). Diday rapporte 20 classes en 3 minutes 30 sur une population de 900 individus et 70 paramètres.
- **Informations complémentaires** : en répétant les tirages, on identifie les individus charnières, la classifiabilité des données, et distingue les formes fortes des formes faibles.
- **Absence de seuil arbitraire** : contrairement aux méthodes de type Rocchio, aucun seuil n'est nécessaire à la formation des classes.

4.2. Faiblesses

- **Dépendance à l'initialisation** : comme pour les k-means, le résultat dépend du tirage initial des étalons. L'algorithme ne converge que vers un *optimum local* et non global.
- **Nécessité de fixer K et n_1** : le nombre de classes et le nombre d'étalons doivent être fixés *a priori*, ce qui suppose une certaine connaissance du problème.
- **Classes vides** : lorsque K est trop grand par rapport au nombre réel de classes, des classes vides apparaissent.
- **Coût de la diagonalisation** : pour les nuées de type « axes » ou « structure », une ACP est effectuée à chaque itération sur chaque classe, ce qui augmente le coût (de l'ordre de $\mathcal{O}(K \cdot M^3)$ par itération).
- **Sensibilité aux valeurs aberrantes** : selon le type de nuée (point, distribution), la présence de valeurs extrêmes peut biaiser la représentation.

- **Absence de garantie globale** : le théorème de convergence suppose une fonction d'agrégation-écartement carrée ; en pratique l'hypothèse n'est pas toujours vérifiée, même si la convergence est observée expérimentalement.
- **Interprétation** : contrairement aux arbres de classification hiérarchique, la méthode ne fournit pas de structure arborescente directement interprétable.

5. Conclusion

La méthode des Nuées Dynamiques, proposée par Edwin Diday en 1971, constitue une avancée majeure en classification automatique et reconnaissance des formes. En généralisant l'idée des k-means — où chaque classe est résumée par un simple centroïde — à une représentation plus riche (ensemble d'étalons, axe factoriel, distribution de probabilité, structure représentative), elle offre un cadre unifié et flexible pour aborder des problèmes variés.

Dans ce travail, nous avons :

1. présenté le contexte historique et conceptuel du Machine Learning et du clustering ;
2. exposé en détail l'algorithme, ses notations et son fondement mathématique, y compris le théorème de convergence ;
3. implémenté entièrement l'algorithme en Python (from scratch, sans bibliothèque externe) ;
4. illustré ses applications (neurologie, marketing, bio-informatique, etc.) ;
5. dressé un bilan des forces et des faiblesses.

Les résultats obtenus confirment la pertinence de la méthode : la représentation par nuée permet de capturer des structures allongées ou courbes que les k-means manqueraient. En répétant les tirages, on identifie les formes « fortes » (stables) et les formes « faibles » (instables), ce qui apporte au praticien une information précieuse sur la classifiabilité des données.

En perspective, on pourrait :

- combiner les Nuées Dynamiques avec des méthodes de deep learning (autoencodeurs pour l'initialisation, prototypes appris) ;
- étendre la méthode au clustering en flux (streaming) ;
- étudier la sélection automatique du nombre de classes K (par exemple via des critères d'information : BIC, AIC, silhouette) ;
- intégrer des contraintes métier (must-link / cannot-link) pour un clustering semi-supervisé.

En définitive, la méthode des Nuées Dynamiques reste, plus de cinquante ans après sa publication, un outil fondamental et toujours pertinent de la classification automatique, dont l'héritage se retrouve dans de nombreux algorithmes modernes.

6. Références bibliographiques

1. E. DIDAY, *Une nouvelle méthode en classification automatique et reconnaissance des formes : la méthode des nuées dynamiques*, Revue de Statistique Appliquée, tome XIX, n°2, 1971, pp. 19–33.
2. S. REGNIER, *Sur quelques aspects mathématiques des problèmes de classification automatique*, I.C.C. Bulletin, vol. 4, 1965, pp. 175–191.
3. E. DIDAY, *La méthode des nuées dynamiques et la reconnaissance des formes*, Fascicule D.I.A., IRIA Rocquencourt, 1971.
4. J.M. ROMEDER, *Méthodes et programmes d'analyse discriminante*, Thèse de 3e cycle, I.S.U.P., Faculté des Sciences de Paris.
5. E. DIDAY, J.C. JACOB, J. PICARD, A. SCHROEDER, *Étude statistique sur une enquête concernant le métabolisme énergétique du cerveau*, Fascicule DIA, IRIA.

7. Annexe : illustration d'exécution

Cette annexe présente une capture d'écran de l'exécution du programme implémenté dans la section 2.2.



```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: G:\BUETUENA_MALOZI-TP_DATA_SCIENCE-ALGO_NUEES_DYNAMIQUES-FROM_SCRATCH.py

MÉTHODE DES NUÉES DYNAMIQUES (E. Diday, 1971)

--- Saisie des données ---
Nombre d'individus N : 5
Nombre de paramètres (dimension) M : 4
Entrez les 5 individus (4 valeur(s) par individu) :
x1 : 17 61 1 0
x2 : 15 64 0 0
x3 : 33 70 4 1
x4 : 18 60 0 0
x5 : 42 74 3 1

--- Paramètres de l'algorithme ---
Nombre de classes K (2 <= K <= 5) : 3

Type de nuée (représentation des classes) :
1. Point unique (équivalent k-means)
2. Ensemble de points (nuée d'étalons)
3. Axes factoriels (1 axe principal)
4. Distribution de probabilité (gaussienne)
5. Structure représentative (sous-espace affine)
Votre choix : 2
Nombre d'étalons nl par classe (1 <= nl <= 5) : 3

Mode d'initialisation :
1. Aléatoire (recommandé)
2. Avec graine fixe (reproductible)
Votre choix : 1

--- Exécution de l'algorithme ---
Itération 1 : S = 16.881967 tailles = [2, 3, 0]
Itération 2 : S = 16.881967 tailles = [2, 3, 0]
>> Convergence atteinte à l'itération 2

RÉSULTATS

--- Classe 1 : 2 individu(s) ---
Indices : [3, 5]
3 étalon(s) :
[33.0, 70.0, 4.0, 1.0]
[42.0, 74.0, 3.0, 1.0]
[18.0, 60.0, 0.0, 0.0]

--- Classe 2 : 3 individu(s) ---
Indices : [1, 2, 4]
3 étalon(s) :
[17.0, 61.0, 1.0, 0.0]
[18.0, 60.0, 0.0, 0.0]
[15.0, 64.0, 0.0, 0.0]

--- Classe 3 : 0 individu(s) ---
Indices : []
3 étalon(s) :
[18.0, 60.0, 0.0, 0.0]
[17.0, 61.0, 1.0, 0.0]
[15.0, 64.0, 0.0, 0.0]

Valeur finale S(L,L) = 16.881967
Nombre d'itérations : 2

>>>
```

FIGURE 1 – Illustration de l'exécution de l'algorithme des Nuées Dynamiques (saisie clavier et résultats).

Sur cette capture, on observe :

- la saisie interactive des individus et de la dimension ;
- le choix du type de nuée (point, ensemble, axes, distribution, structure) ;

- la trace des itérations successives avec la valeur décroissante du critère $S(L, L)$;
- l’affichage final des classes, des nuées et de la convergence.